Table des matières

Cours 09 — Séries temporelles et as-of joins 1
Objectifs 1
Théorie 2
Points clés à retenir 13
Exercices 14

Cours 09 — Séries temporelles et as-of joins

Module 9/12 — Formation SQL avancé

Comment lire ce module — C'est le sommet de la partie A, et le module le plus distinctif de la formation. On y apprend le SQL du temps : mesurer une évolution, lisser sur une durée plutôt que sur un nombre de lignes, découper en périodes, et joindre deux séries qui ne tombent pas aux mêmes instants. La théorie est ici ; les deux laboratoires (`Labo_09_OLAP_SQL.md` et `Labo_09_OLTP_SQL.md`) posent les mêmes questions sur les deux terrains, et c'est l'un des rares modules où la donnée elle-même diffère assez pour que le contraste enseigne quelque chose.

Où s'arrêter — Ce cours fait environ onze heures en tout, un peu plus de quatre séances de deux heures et demie. La théorie et les deux labos en prennent deux, les exercices et leur corrigé le reste. La coupure tombe après la section 6 : les sections 1 à 6 bâtissent une série juste et y rattachent la bonne valeur, les sections 7 à 9 en tirent des sessions, des quantiles et des cohortes. Les labos se coupent au même endroit, à la fin de la partie 4 sur l'entrepôt et de la partie 5 sur le transactionnel.

Objectifs

À la fin de ce module, vous saurez :

- distinguer un instant, une période et un intervalle de validité, et dire quelle erreur produit chaque confusion ;
- reconstituer un axe de dates complet, et expliquer pourquoi une série trouée ment par omission ;
- calculer une variation, un rendement simple et un rendement logarithmique, sans jamais diviser par zéro ;
- choisir entre un cadre en lignes et un cadre en durée, et dire lequel répond à la question posée ;
- dire dans quel fuseau tombe une journée, et reconnaître les journées de 23 et de 25 heures ;
- rééchantillonner une série à la période voulue, y compris une période de largeur libre, et en tirer l'ouverture, le plus haut, le plus bas et la clôture ;

© 2026 iAlexMG. Tous droits réservés.

- écrire un as-of join portable, celui qu'un entretien technique attend, et reconnaître le raccourci natif quand il existe ;
- découper une suite d'événements en sessions par le motif drapeau puis somme cumulée ;
- calculer une médiane et un quantile en sachant lequel des deux opérateurs interpole ;
- construire une cohorte et lire une rétention.

Théorie

1. Le grain temporel

Labo : `Labo_09_OLAP_SQL.md` et `Labo_09_OLTP_SQL.md`, partie 1.

Trois choses portent des dates, et on les confond constamment. Les distinguer coûte une minute et évite trois classes d'erreurs.

La chose	Ce qu'elle est	Exemple sur le cas d'étude	La colonne typique
Un instant	un point sur l'axe, sans durée	la commande a été passée le 12 mars à 14 h 03	`orderdate`
Une période	un seau qui a un début et une fin, et qui pave l'axe sans trou ni recouvrement	mars 2024, la semaine 12, la journée du 12 mars	`DateKey` de `dbo.DimDate`
Un intervalle de validité	un état vrai depuis une date, jusqu'à une autre ou jusqu'à nouvel ordre	ce produit valait 34,99 $ du 1er juin 2023 au 30 mai 2024	`startdate` et `enddate`

Les erreurs que chaque confusion produit sont distinctes.

Confondre un instant et une période donne le classique du décalage d'un jour. Filtrer `orderdate <= '2024-03-31'` sur une colonne horodatée écarte tout ce qui est arrivé le 31 mars après minuit, donc presque toute la journée. La parade est de raisonner en intervalle semi-ouvert : `orderdate >= '2024-03-01' AND orderdate < '2024-04-01'`. Cette écriture est aussi la seule qui reste juste quand la colonne change de type, et la seule que le Cours 10 saura indexer.

Confondre une période et un intervalle de validité donne le double comptage. Une période pave l'axe : chaque instant appartient à une seule. Un intervalle de validité ne garantit rien de tel, et deux versions qui se chevauchent multiplient les lignes d'une jointure sans qu'aucune erreur ne soit levée.

Confondre un instant et un intervalle de validité donne l'anachronisme, l'erreur la plus coûteuse des trois. On rattache à une commande de 2023 le prix du produit tel qu'il est aujourd'hui, et le rapport de marge de l'an dernier change chaque fois qu'on le rejoue. La section 6 traite ce cas en entier.

© 2026 iAlexMG. Tous droits réservés.

Une période se dit aussi dans un fuseau. Le Cours 02 a montré qu'un instant change de jour selon le fuseau où on le lit : une série quotidienne bâtie sur des instants dit donc dans quel fuseau tombent ses journées, `CAST(instant AT TIME ZONE 'America/Toronto' AS DATE)`, faute de quoi elle dépend de la machine qui la calcule. Et dans un fuseau qui change d'heure, deux journées par an ne durent pas 24 heures. À Montréal, le 8 mars 2026 en compte 23 et le 1er novembre 25 : une série horaire de ces journées-là, générée d'instant en instant, rend 23 et 25 points, mesuré des deux côtés, et celle du 1er novembre n'a pourtant que 24 heures de mur distinctes, parce que 1 h revient deux fois.

```sql
SELECT COUNT(*) AS heures_du_1er_novembre
FROM   generate_series(TIMESTAMP '2026-11-01 00:00:00' AT TIME ZONE 'America/Toronto',
                       TIMESTAMP '2026-11-02 00:00:00' AT TIME ZONE 'America/Toronto' - INTERVAL '1 hour',
                       INTERVAL '1 hour') AS g(h);
```

Trois conséquences pour qui travaille une série. Une moyenne horaire qui divise le total du jour par 24 se trompe deux fois par an. Une série rangée en heure de mur a un trou au printemps et un doublon à l'automne, celui que le Cours 02 a montré la nuit du 1er novembre. Et une journée ne vaut pas 24 heures dans un calcul : dans une session à l'heure de l'Est, `+ INTERVAL '1 day'` ajouté au 1er novembre à minuit tombe sur le 2 novembre à minuit, `+ INTERVAL '24 hours'` sur le 1er à 23 h, sur les deux moteurs. La règle qui évite tout cela : ranger des instants, et ne passer à l'heure de mur qu'au moment de regrouper par jour ou d'afficher.

Portabilité — La durée de cette journée ne se mesure pas pareil sur nos deux terrains. La soustraction de ses deux minuits rend `1 day 01:00:00` sur PostgreSQL, et `1 day` sur DuckDB, qui compte en jours de calendrier ; celle du 8 mars, `23:00:00` contre `1 day`. Pour obtenir les heures écoulées sur l'entrepôt, `DATE_DIFF('hour', debut, fin)` rend 25, et la différence des `EPOCH` aussi, mesuré. Une durée qui compte se calcule donc en secondes, `EXTRACT(EPOCH FROM …)`, la seule forme qui donne le même nombre partout.

À retenir — Un intervalle semi-ouvert, fermé à gauche et ouvert à droite, est le seul découpage qui ne perd ni ne double une ligne. Écrivez `>= début AND < fin`, jamais `BETWEEN` sur une colonne horodatée.

Rappel — Le `BETWEEN` qui perd la dernière journée d'un horodatage est démontré au Cours 02, section 2.

2. L'axe complet

Labo : `Labo_09_OLAP_SQL.md`, sous-parties 4.1 et 4.2 ; `Labo_09_OLTP_SQL.md`, sous-parties 2.1 et 2.2.

Une série qui vient d'un `GROUP BY` ne contient que les périodes où il s'est passé quelque chose. Les autres n'existent pas, et leur absence ne se voit pas.

C'est le geste du Cours 05 : on génère l'axe, puis on y raccroche les mesures par un `LEFT JOIN`.

```sql
-- PostgreSQL : l'axe des jours, puis les ventes raccrochées.
WITH axe AS (
    SELECT jour::date AS jour
    FROM   generate_series(DATE '2023-01-01', DATE '2023-12-31', INTERVAL '1 day') AS jour
)
SELECT a.jour,
       COALESCE(SUM(d.linetotal), 0) AS ventes
```


© 2026 iAlexMG. Tous droits réservés.

```sql
FROM   axe AS a
LEFT   JOIN sales.salesorderheader AS h ON h.orderdate >= a.jour
                                       AND h.orderdate <  a.jour + 1
LEFT   JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
GROUP  BY a.jour
ORDER  BY a.jour;
```

Rappel — Les littéraux `DATE '…'` et `INTERVAL '…'`, et ce que rend leur addition, sont présentés au Cours 02, section 6.

Sur l'entrepôt, l'axe existe déjà : c'est `dbo.DimDate`, et c'est l'une des raisons d'être d'une dimension de date. On la filtre sur la plage voulue et on y raccroche le fait.

Le `COALESCE` mérite un mot. Sans lui, un jour sans vente porte `NULL` plutôt que zéro, et la différence n'est pas cosmétique : une moyenne ignore les `NULL` et les compte pour rien au dénominateur, alors qu'elle inclut les zéros. La moyenne d'une série trouée n'est donc pas la moyenne de la période, elle est la moyenne des jours actifs, ce qui est une autre question. Les deux réponses sont légitimes ; ce qui ne l'est pas, c'est de ne pas savoir laquelle on a calculée.

Un axe complet n'est pas seulement une question d'affichage. Toute la section 4 en dépend, et la section 9 aussi.

3. Variation et rendement

Labo : `Labo_09_OLAP_SQL.md`, sous-partie 4.5.

Le `LAG` du Cours 04 sert ici pour de bon.

```sql
SELECT jour,
       ventes,
       ventes - LAG(ventes) OVER (ORDER BY jour)                    AS variation,
       (ventes - LAG(ventes) OVER (ORDER BY jour))
         / NULLIF(LAG(ventes) OVER (ORDER BY jour), 0)              AS variation_relative
FROM   serie_quotidienne;
```

Le `NULLIF` est la division protégée : il transforme un dénominateur nul en `NULL`, et le résultat devient `NULL` au lieu de faire tomber la requête. Sur une série complétée par des zéros, un jour sans vente suivi d'un jour actif produirait une division par zéro à coup sûr.

La variation relative porte un nom dans le monde financier, le rendement simple, et elle a un défaut que tout le monde découvre une fois. Les rendements simples ne s'additionnent pas. Une hausse de 50 % suivie d'une baisse de 50 % ne ramène pas au point de départ, elle laisse à 75 % de la valeur initiale, parce que les deux pourcentages ne portent pas sur la même base.

Le rendement logarithmique corrige exactement cela.

```sql
SELECT jour,
       LN(ventes / NULLIF(LAG(ventes) OVER (ORDER BY jour), 0)) AS rendement_log
FROM   serie_quotidienne
WHERE  ventes > 0;
```


© 2026 iAlexMG. Tous droits réservés.

Le logarithme transforme un produit en somme : le rendement logarithmique d'une période longue est la somme des rendements logarithmiques des périodes courtes qui la composent. C'est ce qui permet de cumuler, de faire des moyennes, et de calculer un écart type qui veut dire quelque chose. Le prix à payer est le domaine de définition, puisque le logarithme refuse zéro et refuse le négatif, d'où le filtre.

Rappel — `LN`, `EXP` et `POWER` sont présentés au Cours 02, section 4.

Portabilité — `LN` a le même sens sur les deux moteurs. `LOG` est plus glissant : sur PostgreSQL comme sur l'entrepôt, `LOG(x)` est le logarithme en base 10, et l'entrepôt accepte en plus la forme à deux arguments, `LOG(10, 100)`, qui rend la même chose. Ailleurs, la forme à deux arguments est la seule, dont sur Snowflake d'après sa documentation, ce qui change le sens d'un appel à un seul argument. Écrivez `LN` quand vous voulez le logarithme naturel, sur les deux terrains, et vous n'aurez pas à vous en souvenir.

4. Les cadres en temps

Labo : `Labo_09_OLAP_SQL.md`, sous-partie 4.3 ; `Labo_09_OLTP_SQL.md`, partie 3.

Voici le moment où le cadre cesse d'être un piège et devient l'outil central du module. Le Cours 04 vous a dit d'écrire `ROWS` parce que c'est prévisible, et c'était le bon conseil à ce moment-là. Cette section dit quand ce conseil ne suffit plus.

`ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` compte six lignes physiques en arrière. `RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW` prend toutes les lignes dont la valeur d'`ORDER BY` tombe dans les six derniers jours. Sur une série complète, une ligne par jour, les deux donnent le même résultat. Sur une série trouée, elles divergent, et aucune des deux ne répond à toutes les questions.

La série contient	**`ROWS 6 PRECEDING`**	**`RANGE INTERVAL '6 days' PRECEDING`**
un jour sur deux	remonte douze jours en arrière	remonte six jours, et n'y trouve que trois ou quatre lignes
deux lignes le même jour	en compte deux, et ne remonte que cinq autres lignes	les prend toutes les deux, la durée reste juste
aucune ligne certains jours	ignore les jours vides et compense en remontant plus loin	reste dans les sept jours, mais une moyenne n'y divise que par les jours présents

La question à se poser est toujours la même : voulez-vous la moyenne des sept derniers jours, ou la moyenne des sept dernières observations ? Sur un contrôle de qualité qui suit les sept derniers relevés d'un capteur, c'est le nombre d'observations qui compte, donc le cadre en lignes. Sur un indicateur de tendance, c'est la durée, et le cadre en durée y suffit pour une somme. Pour une moyenne quotidienne, il ne suffit pas : un jour sans ligne n'y compte ni pour zéro ni pour rien, il disparaît du dénominateur. Mesuré des deux côtés sur trois ventes, 10 $ le 1er mars, 20 $ le 2 et 30 $ le 7 : au 7, le cadre en durée rend 20 $, la moyenne des trois ventes, alors que la moyenne quotidienne des sept jours vaut 60 $ divisés par sept, 8,57 $.

© 2026 iAlexMG. Tous droits réservés.

Une troisième réponse existe, et c'est souvent la meilleure : compléter l'axe comme à la section 2, puis employer `ROWS`. Sur une série sans trou, les deux cadres coïncident par construction, les jours vides comptent pour zéro, et vous n'avez plus à espérer que le moteur d'en face supporte votre écriture. C'est aussi la seule des trois qui donne la moyenne quotidienne.

Portabilité — Le cadre en durée passe sur nos deux terrains : `RANGE BETWEEN INTERVAL 2 DAY PRECEDING AND CURRENT ROW` y donne bien une somme glissante sur trois jours, mesuré des deux côtés. PostgreSQL l'accepte depuis sa version 11, et ajoute un troisième mode, `GROUPS`, qui compte les groupes de valeurs égales plutôt que les lignes. Ailleurs, ce cadre est plus restreint et son support a bougé dans le temps, dont sur Snowflake d'après sa documentation. Préférez donc la parade universelle dans du code qui voyage : complétez l'axe, puis écrivez `ROWS`. C'est l'écriture qui passe partout et qu'un entretien accepte sans discussion.

Rappel — Le cadre par défaut reste celui du Cours 04. Dès qu'un `ORDER BY` figure dans le `OVER` sans cadre explicite, le moteur applique `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, et les valeurs d'ordre en double sont additionnées d'un coup. Sur une série temporelle qui porte plusieurs lignes le même jour, cette valeur par défaut n'est pas neutre.

5. Rééchantillonner

Labo : `Labo_09_OLAP_SQL.md`, sous-partie 4.4 ; `Labo_09_OLTP_SQL.md`, sous-partie 2.4.

Rééchantillonner, c'est passer d'un grain fin à un grain plus large. La brique de base est `DATE_TRUNC`, qui existe des deux côtés et se lit sans effort : elle rabat une date sur le début de sa période.

```sql
SELECT DATE_TRUNC('month', h.orderdate) AS mois,
       SUM(d.linetotal)                 AS ventes
FROM   sales.salesorderheader AS h
JOIN   sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
GROUP  BY 1
ORDER  BY 1;
```

Rappel — `DATE_TRUNC`, et le type qu'il rend sur chaque moteur, sont présentés au Cours 02, section 6.

Deux précautions. D'abord, le résultat reste un horodatage, pas une étiquette : `2024-03-01` désigne le mois de mars, et l'afficher tel quel suffit dans un rapport technique, moins dans un rapport métier. Ensuite, la semaine n'a pas le même premier jour partout. PostgreSQL suit ISO 8601 et fait commencer la semaine le lundi, et l'entrepôt fait de même : `DATE_TRUNC('week', DATE '2024-03-17')` rend le lundi 11 mars des deux côtés, alors que le 17 est un dimanche. D'autres moteurs laissent un paramètre de session en décider, `WEEK_START` sur Snowflake d'après sa documentation, ce qui veut dire qu'un rapport hebdomadaire peut y changer de valeur sans qu'une ligne de SQL ait bougé. Sur un rapport hebdomadaire qui compte, fixez le paramètre explicitement, ou tronquez à la main.

`DATE_TRUNC` ne connaît que les unités du calendrier : l'heure, le jour, la semaine, le mois. Une période de largeur libre, le quart d'heure d'un compteur, la quinzaine d'une paie, demande une autre fonction, et c'est l'un des rares points où nos deux terrains n'ont pas le même mot. DuckDB écrit `TIME_BUCKET(INTERVAL '14 days', ts)`, PostgreSQL `DATE_BIN(INTERVAL '14 days', ts, origine)`, et chacun ignore la fonction de l'autre. Une période de largeur libre se compte à partir d'une origine, et c'est là que les deux divergent sans prévenir :

© 2026 iAlexMG. Tous droits réservés.

Écriture	Le 17 mars 2024 tombe dans la quinzaine du	Mesuré sur
`TIME_BUCKET(INTERVAL '14 days', ts)`	lundi 11 mars	DuckDB, origine implicite au lundi 3 janvier 2000
`DATE_BIN(INTERVAL '14 days', ts, TIMESTAMP '2000-01-01')`	samedi 9 mars	PostgreSQL
`DATE_BIN(INTERVAL '14 days', ts, TIMESTAMP '2000-01-03')`	lundi 11 mars	PostgreSQL, qui retombe sur DuckDB

La même largeur, deux origines, deux découpages : un rapport par quinzaine porté d'un moteur à l'autre change de bornes et de chiffres sans qu'une erreur ne sorte. Écrivez l'origine, toujours : `TIME_BUCKET` l'accepte en troisième argument, et `DATE_BIN` l'exige. Les deux diffèrent encore sur une dernière chose : `TIME_BUCKET` accepte une largeur en mois, `DATE_BIN` la refuse (« ne peuvent pas être rangées dans des intervalles contenant des mois ou des années »), et c'est `DATE_TRUNC` qui reprend la main pour le mois.

Portabilité — Snowflake a sa propre fonction, `TIME_SLICE`, d'après sa documentation, et l'extension TimescaleDB de PostgreSQL ajoute un `TIME_BUCKET`, qui n'est pas celui du moteur nu. Une période libre est donc la ligne du module qui voyage le moins : nommez la fonction et l'origine en commentaire de la requête, pour que celui qui la portera sache quoi chercher.

Une fois la période choisie, on en tire quatre mesures qui reviennent partout : la première valeur, la dernière, la plus haute et la plus basse. Le monde financier les appelle l'ouverture, la clôture, le plus haut et le plus bas ; un rapport de ventes y verra le premier jour, le dernier jour, le pic et le creux. C'est la même requête.

```sql
-- PostgreSQL : les quatre mesures d'une période, sans fonction de fenêtrage.
SELECT DATE_TRUNC('month', jour)                AS mois,
       MIN(ventes)                              AS plus_bas,
       MAX(ventes)                              AS plus_haut,
       (ARRAY_AGG(ventes ORDER BY jour))[1]     AS ouverture,
       (ARRAY_AGG(ventes ORDER BY jour DESC))[1] AS cloture
FROM   serie_quotidienne
GROUP  BY 1
ORDER  BY 1;
```

L'ouverture et la clôture demandent un mot, parce qu'un agrégat ordinaire ne sait pas les calculer : `MIN` et `MAX` répondent sur la valeur, pas sur la position. Deux écritures existent. La première passe par une fonction de fenêtrage, `FIRST_VALUE` et `LAST_VALUE`, et rencontre alors le piège du Cours 04 : sans cadre explicite, `LAST_VALUE` regarde jusqu'à la ligne courante et renvoie donc la ligne courante, ce qui n'est jamais ce qu'on voulait. Il faut écrire `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`. La seconde écriture passe par un agrégat ordonné, comme ci-dessus. L'entrepôt en offre une troisième, la plus lisible des trois : `MIN_BY` et `MAX_BY` répondent littéralement à « la valeur de x pour la ligne où y est le plus petit », sans fenêtre ni cadre à surveiller. PostgreSQL ne les a pas, et c'est l'un des rares endroits où la voie analytique écrit plus court sans rien perdre.

6. L'as-of join

© 2026 iAlexMG. Tous droits réservés.

Labo : `Labo_09_OLAP_SQL.md`, partie 3 ; `Labo_09_OLTP_SQL.md`, partie 4.

C'est le cœur du module, et l'outil qui distingue un SQL d'analyste d'un SQL de débutant.

Le problème se pose dès que deux séries ne tombent pas aux mêmes instants. Vous avez des événements d'un côté, des commandes par exemple, et de l'autre une valeur qui change de temps en temps, un prix de liste ou un taux. Il n'existe aucune égalité entre les deux : la commande du 12 mars ne correspond à aucune ligne datée du 12 mars dans l'historique des prix. La question est « quelle était la dernière valeur connue au moment de l'événement », et l'égalité ne sait pas l'exprimer.

L'écriture portable est celle du Cours 07, employée pour la troisième fois.

```sql
-- Le prix de liste en vigueur à la date de la commande.
SELECT h.salesorderid,
       h.orderdate,
       d.productid,
       prix.listprice AS prix_du_jour
FROM   sales.salesorderheader AS h
JOIN   sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
LEFT   JOIN LATERAL (
         SELECT p.listprice
         FROM   production.productlistpricehistory AS p
         WHERE  p.productid  = d.productid
           AND  p.startdate <= h.orderdate
         ORDER  BY p.startdate DESC
         LIMIT  1
       ) AS prix ON true;
```

Lisez la sous-requête comme une phrase : parmi les prix de ce produit qui ont commencé avant la commande, prenez le plus récent. Le `LEFT` garde les lignes sans correspondance, celles dont le produit n'a aucun prix antérieur à la commande, et c'est exactement ce qu'on veut voir plutôt que de les perdre en silence.

Trois détails font la différence entre une écriture qui marche et une écriture qui trompe.

Le tri décroissant plus `LIMIT 1` est ce qui donne « la dernière », et il doit porter sur la même colonne que la comparaison. Trier sur une autre colonne, la date de modification technique par exemple, donne un résultat plausible et faux.

La comparaison est `<=` et non `<`. Une valeur qui entre en vigueur le jour même de l'événement s'applique à l'événement. C'est une convention, mais c'est la convention usuelle, et elle doit être écrite quelque part plutôt que subie.

Et surtout, la condition ne mentionne pas la date de fin. C'est la règle qui sauve, et elle mérite son encadré.

À retenir — L'as-of join s'appuie sur la date de début, jamais sur l'intervalle fermé. On prend la dernière version dont le début est antérieur ou égal à la date visée. Cette écriture reste juste quand les dates de fin sont absentes, décalées d'un jour, ou franchement incohérentes, et c'est précisément le cas de la donnée que vous allez rencontrer au laboratoire OLAP.

© 2026 iAlexMG. Tous droits réservés.

Le contraste entre les deux terrains est instructif, et il n'est pas qu'une affaire de syntaxe. Le monde transactionnel range l'historique dans une table dédiée que l'application tient à jour, ici `production.productlistpricehistory`, dont les intervalles sont propres. L'entrepôt, lui, replie l'historique dans la dimension elle-même : `dbo.DimProduct` porte plusieurs lignes pour un même produit, avec `StartDate`, `EndDate` et `Status`, ce qu'on appelle une dimension à évolution lente. Le repliement est l'endroit où les erreurs d'alimentation se logent, et cette base en porte une belle, puisque pour plusieurs versions historiques la date de fin précède la date de début de quatre ans.

Un `WHERE date_de_vente BETWEEN StartDate AND EndDate` ne renverrait donc jamais rien pour ces versions. Pas d'erreur, pas d'avertissement, un résultat vide. Vous reconnaissez la famille : c'est le `NOT IN` sur colonne nullable du Cours 07 et la valeur `JSON` comparée sans la flèche double du Cours 08. La faute qui coûte cher ne lève pas d'erreur.

Portabilité — L'entrepôt offre un raccourci natif, et il tourne :

```
SELECT e.d AS evenement, p.d AS prix_du, p.p AS prix
FROM   evenement AS e
ASOF LEFT JOIN historique AS p ON p.d <= e.d;
```

Une clause au lieu d'une sous-requête corrélée, et le moteur l'exécute mieux qu'une boucle de `LATERAL`. Le `LEFT` joue son rôle habituel : sans lui, un événement antérieur à toute valeur connue disparaîtrait au lieu de ressortir avec des `NULL`. D'autres plateformes ont le même raccourci sous une forme plus verbeuse, `ASOF JOIN … MATCH_CONDITION (…) ON (…)` sur Snowflake d'après sa documentation ; PostgreSQL n'a rien de tel et reste au `LATERAL`.

La réponse à donner en entretien reste malgré tout l'écriture portable : elle fonctionne des deux côtés et montre que vous avez compris le mécanisme plutôt que retenu un mot-clé. Écrivez le raccourci quand vous êtes chez vous, sachez écrire l'autre quand on vous regarde.

Le geste dépasse largement le prix de liste. Le dernier cours coté au moment de l'ordre, le taux de change en vigueur au moment de la facture, la version du tarif applicable au moment de l'abonnement, la valeur du capteur la plus récente au moment de l'alarme : c'est la même requête à chaque fois. C'est pour cette raison qu'un moteur analytique finit toujours par offrir un `ASOF JOIN`.

7. Îlots et lacunes

Labo : `Labo_09_OLAP_SQL.md`, sous-partie 5.2 ; `Labo_09_OLTP_SQL.md`, sous-partie 6.1.

Une suite d'événements se découpe naturellement en groupes dès qu'un écart dépasse un seuil. Les groupes s'appellent des îlots, les vides entre eux des lacunes, et le motif de résolution est toujours le même : un drapeau, puis une somme cumulée.

```
WITH marque AS (
    SELECT customerid,
           orderdate,
           CASE WHEN orderdate - LAG(orderdate) OVER (PARTITION BY customerid
                                                      ORDER BY orderdate) > INTERVAL '30 days'
                THEN 1 ELSE 0 END AS rupture
    FROM   sales.salesorderheader
)
```


© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT customerid,
       orderdate,
       SUM(rupture) OVER (PARTITION BY customerid
                          ORDER BY orderdate
                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS session
FROM   marque
ORDER  BY customerid, orderdate;
```

Le drapeau vaut 1 quand l'écart avec l'événement précédent dépasse le seuil. La somme cumulée de ce drapeau incrémente donc un compteur à chaque rupture et le laisse tranquille le reste du temps : toutes les lignes d'un même îlot partagent la même valeur, et cette valeur est un identifiant de session. Le premier événement d'un client porte `NULL` en `LAG`, donc `0` en drapeau, ce qui le place correctement dans la session zéro.

Une variante du même motif traite les suites d'entiers ou de dates consécutives. On soustrait le numéro de ligne à la valeur : tant que les valeurs se suivent, la différence est constante, et elle change au premier trou. Grouper par cette différence donne les îlots directement. C'est l'un des rares endroits du SQL où l'astuce est vraiment une astuce, et c'est une question d'entretien classique. Elle est plus courte à écrire que le drapeau, et nettement moins lisible : sur du code qui doit vivre, le drapeau reste préférable.

8. Quantiles

Labo : `Labo_09_OLAP_SQL.md`, sous-partie 5.1.

Un quantile coupe une distribution triée en parts. Le SQL en offre deux familles, et elles ne répondent pas à la même question.

`NTILE(n)`, vu au Cours 04, est une fonction de fenêtrage : elle attribue un numéro de seau à chaque ligne. Elle répond à « dans quel décile se trouve ce client ». Elle répartit les lignes le plus également possible, ce qui a une conséquence désagréable : deux lignes de même valeur peuvent atterrir dans deux seaux différents, parce que la fonction compte des lignes et ignore les égalités.

`PERCENTILE_CONT` et `PERCENTILE_DISC` sont des agrégats à ordre, écrits avec `WITHIN GROUP`. Ils répondent à « quelle est la valeur qui coupe la distribution à cet endroit ».

```sql
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY totaldue) AS mediane_interpolee,
       PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY totaldue) AS mediane_observee,
       PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY totaldue) AS decile_9
FROM   sales.salesorderheader;
```

La différence entre les deux tient en un mot. `PERCENTILE_DISC` renvoie une valeur qui existe vraiment dans les données : sur quatre montants, la médiane discrète est l'un des quatre. `PERCENTILE_CONT` interpole entre les deux valeurs qui encadrent le point de coupe, et peut donc renvoyer un nombre que personne n'a jamais observé.

Interpoler est légitime quand la grandeur est continue et que les valeurs intermédiaires ont un sens : un montant, une durée, une température. Ce ne l'est pas quand la grandeur est discrète ou catégorielle. La médiane interpolée d'un nombre d'enfants par ménage vaut 1,5, ce qui ne décrit aucun ménage. La médiane interpolée d'un identifiant de produit est une absurdité. La règle : si la moyenne a un sens, l'interpolation en a un aussi ; sinon, prenez la version discrète.

© 2026 iAlexMG. Tous droits réservés.

Astuce — L'entrepôt offre `MEDIAN(x)`, strictement équivalent à `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x)` : les deux rendent 364,09 $ sur les prix du catalogue, vérifié. PostgreSQL n'a pas de `MEDIAN` et demande la forme longue. Les grandes plateformes ajoutent des variantes approchées, `APPROX_PERCENTILE` sur Snowflake d'après sa documentation, qui échangent un peu d'exactitude contre beaucoup de vitesse sur de gros volumes.

9. Cohortes

Labo : `Labo_09_OLAP_SQL.md`, sous-partie 5.3 ; `Labo_09_OLTP_SQL.md`, sous-partie 6.2.

Une cohorte regroupe les entités selon la date de leur premier événement, puis suit ce groupe dans le temps. C'est l'outil qui répond à « nos clients de 2023 reviennent-ils mieux que ceux de 2022 », et il se construit en trois gestes.

Le premier geste est la date de première occurrence par entité, un simple `MIN` par groupe.

```sql
WITH premiere AS (
    SELECT customerid,
           MIN(DATE_TRUNC('month', orderdate)) AS mois_cohorte
    FROM   sales.salesorderheader
    GROUP  BY customerid
),
activite AS (
    SELECT h.customerid,
           p.mois_cohorte,
           DATE_TRUNC('month', h.orderdate) AS mois_actif
    FROM   sales.salesorderheader AS h
    JOIN   premiere AS p ON p.customerid = h.customerid
)
SELECT mois_cohorte,
       (EXTRACT(YEAR  FROM mois_actif) - EXTRACT(YEAR  FROM mois_cohorte)) * 12
     + (EXTRACT(MONTH FROM mois_actif) - EXTRACT(MONTH FROM mois_cohorte)) AS rang_mois,
       COUNT(DISTINCT customerid) AS clients
FROM   activite
GROUP  BY 1, 2
ORDER  BY 1, 2;
```

Le deuxième geste est l'écart en périodes entre le mois d'activité et le mois de la cohorte. Le calcul ci-dessus est explicite volontairement : il compte des mois calendaires et il est portable. L'entrepôt écrirait `DATEDIFF('month', mois_cohorte, mois_actif)`, plus court, qui compte les frontières franchies et non les durées écoulées (Cours 02, section 6). PostgreSQL n'a pas cette fonction, d'où la forme explicite retenue ici.

Le troisième geste est la rétention elle-même, qui est une part : le compte du rang N divisé par le compte du rang zéro, ce dernier étant obtenu par une fonction de fenêtrage ou par une jointure sur la même table. Un tableau croisé du résultat, une ligne par cohorte et une colonne par rang, est le `PIVOT` du Cours 06.

© 2026 iAlexMG. Tous droits réservés.

Deux pièges guettent. Le premier est l'axe incomplet de la section 2 : une cohorte qui n'a aucun client actif au rang 3 n'apparaît pas, et la courbe de rétention saute le point au lieu d'afficher un zéro. Le second est la cohorte trop jeune : la dernière cohorte du jeu de données n'a pas encore eu le temps d'atteindre le rang 6, et sa rétention à ce rang n'est pas mauvaise, elle est inconnue. Afficher un zéro plutôt qu'une case vide sur ce point est un mensonge par présentation.

10. Sur les deux terrains

Labo : `Labo_09_OLAP_SQL.md`, partie 6 ; `Labo_09_OLTP_SQL.md`, partie 7.

Sujet	PostgreSQL (voie OLTP)	DuckDB (voie OLAP)
Générer un axe de dates	`generate_series` sur des dates	`dbo.DimDate`, déjà là, ou `generate_series`, ou la CTE récursive
Où vit l'historique	table dédiée `production.productlistpricehistory`, intervalles propres	replié dans `dbo.DimProduct`, dimension à évolution lente, dates incohérentes
As-of join	`LEFT JOIN LATERAL (… ORDER BY … DESC LIMIT 1) ON true`	la même écriture, plus `ASOF LEFT JOIN` en natif
Cadre en durée	`RANGE BETWEEN INTERVAL … PRECEDING`, plus le mode `GROUPS`	`RANGE BETWEEN INTERVAL … PRECEDING` aussi
Premier jour de la semaine	lundi, fixé par ISO 8601	lundi aussi
Médiane	`PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x)`	idem, plus le raccourci `MEDIAN`
Première et dernière valeur d'une période	`FIRST_VALUE` et `LAST_VALUE` avec cadre explicite, ou un agrégat ordonné	idem, plus `MIN_BY` et `MAX_BY`
Écart entre deux dates	une soustraction, qui rend un intervalle	une soustraction aussi, ou `DATEDIFF('unite', a, b)`
La journée d'un changement d'heure	la soustraction rend `1 day 01:00:00` ou `23:00:00`	la soustraction rend `1 day` ; `DATE_DIFF('hour', …)` rend 25 ou 23
Une période de largeur libre	`DATE_BIN`, origine obligatoire, pas de mois	`TIME_BUCKET`, origine implicite au 3 janvier 2000, mois acceptés
Logarithme naturel	`LN(x)`, et `LOG(x)` est en base 10	`LN(x)`, et `LOG(x)` en base 10, la forme à deux arguments étant acceptée en plus

© 2026 iAlexMG. Tous droits réservés.

Ce tableau penche d'un côté, et il faut le dire : sur les séries temporelles, la voie analytique écrit plus court presque partout. `ASOF LEFT JOIN`, `MEDIAN`, `MIN_BY` et `MAX_BY` remplacent chacun trois à six lignes de la voie transactionnelle, et aucun ne fait perdre en clarté. Les deux lignes qui semblaient opposer les terrains, le cadre en durée et le premier jour de la semaine, ne les opposent pas : ils répondent pareil, mesuré des deux côtés.

La conclusion n'est pas que l'un vaut mieux que l'autre, c'est que chaque moteur accumule les raccourcis de ce qu'on lui demande souvent. On demande des séries temporelles à un entrepôt, alors il en a. Ce que vous devez savoir, c'est écrire les deux formes, et reconnaître laquelle vous avez le droit d'employer là où vous êtes.

Le piège partagé est celui de la section 6, et il ne dépend d'aucun moteur : un intervalle de validité qu'on n'a pas contrôlé produit un résultat vide plutôt qu'une erreur.

Le même travail, deux fois — Les deux laboratoires posent les mêmes questions. `Labo_09_OLTP_SQL.md` travaille sur un historique de prix propre, ce qui laisse voir le mécanisme de l'as-of join sans bruit. `Labo_09_OLAP_SQL.md` pose la même question sur une dimension à évolution lente dont les dates sont cassées, et commence donc par un contrôle de qualité. Faites-les dans cet ordre si vous le pouvez.

Points clés à retenir

- Un instant, une période et un intervalle de validité sont trois choses différentes. Les confondre produit respectivement un décalage d'un jour, un double comptage et un anachronisme.
- Un intervalle semi-ouvert, `>= début AND < fin`, est le seul découpage qui ne perd ni ne double une ligne. `BETWEEN` sur une colonne horodatée ampute la dernière journée.
- Une journée se dit dans un fuseau, et deux par an durent 23 et 25 heures. On range des instants, et on ne passe à l'heure de mur qu'au regroupement ou à l'affichage ; une durée qui compte se calcule en secondes.
- Une série issue d'un `GROUP BY` ne contient que les périodes actives. On génère l'axe, on raccroche par `LEFT JOIN`, et on décide explicitement si un trou vaut zéro ou rien.
- `NULLIF` au dénominateur est la division protégée. Sur une série complétée par des zéros, elle n'est pas optionnelle.
- Les rendements simples ne s'additionnent pas, les rendements logarithmiques oui. Le prix du logarithme est qu'il refuse zéro et le négatif.
- `ROWS` compte des observations, `RANGE` avec un intervalle compte une durée. Sur une série trouée, les deux diffèrent, et une moyenne en durée ne divise que par les jours présents.
- La parade portable du cadre en durée : compléter l'axe, puis écrire `ROWS`. Sur une série sans trou, les deux cadres coïncident par construction, et c'est la seule écriture de la moyenne quotidienne.

© 2026 iAlexMG. Tous droits réservés.

- `DATE_TRUNC` rééchantillonne, mais le premier jour de la semaine n'est pas le même partout. Sur un rapport hebdomadaire qui compte, fixez-le.
- Une période de largeur libre s'écrit `TIME_BUCKET` sur DuckDB, `DATE_BIN` sur PostgreSQL. Leurs origines par défaut diffèrent : écrivez-la toujours.
- `LAST_VALUE` sans cadre explicite renvoie la ligne courante. Le piège du Cours 04 se réveille dès qu'on calcule une clôture de période.
- L'as-of join joint deux séries qui ne coïncident pas. L'écriture portable est `LEFT JOIN LATERAL (… WHERE debut <= evenement ORDER BY debut DESC LIMIT 1) ON true`.
- L'as-of join s'appuie sur la date de début, jamais sur l'intervalle fermé. Cette écriture survit à des dates de fin absentes ou incohérentes.
- Un intervalle de validité qu'on n'a pas contrôlé est une faute silencieuse : la requête renvoie vide, pas une erreur. Troisième membre de la famille, après le `NOT IN` du Cours 07 et la valeur `JSON` comparée sans la flèche double du Cours 08.
- Îlots et lacunes se résolvent par un drapeau puis une somme cumulée. Toutes les lignes d'un îlot partagent la même somme, qui devient l'identifiant de session.
- `NTILE` répartit des lignes et sépare les valeurs égales. `PERCENTILE_CONT` interpole, `PERCENTILE_DISC` renvoie une valeur observée. Si la moyenne a un sens, l'interpolation aussi.
- Une cohorte se construit en trois gestes : première date par entité, écart en périodes, part rapportée au rang zéro. Une cohorte trop jeune a une rétention inconnue, pas nulle.

Exercices

Les exercices de ce module et leur corrigé vivent dans `Exercices_corriges_09_SQL.md`. Faites-les après les deux laboratoires : plusieurs énoncés renvoient à des résultats que vous n'obtiendrez qu'en ayant vu la dimension cassée de vos propres yeux.

© 2026 iAlexMG. Tous droits réservés.